Laith Alneemy 24958454

Individual Reflection

What's the worst case for your algorithm, and what kind of input triggers it?

The worst case time complexity is O(n) for search. Insert and erase are always O(k) where k is the max height, which is bounded by 32. The worst case occurs when heights are not distributed randomly, essentially resulting in a singly linked list. This could occur if the heights are mostly equivalent, or if the skip list has a descending staircase structure. Memory is technically always O(n), but is much worse if all nodes are given the max height, as this results in 32 times the memory allocation for storing the next pointers, compared to one pointer being stored when the height is only 1, as is the most common case.

Input doesn't matter for creating these bad cases assuming the random number generator isn't being exploited. These bad cases are not feasibly possible for large skip trees due to their probabilistic unlikelihood. 50 nodes having the same height of 1 would be $\frac{1}{2^{50}}$. Input can matter in cases where the random number generator is exploited. If someone is able to know the heights in advance, because the random number generator doesn't contain a source of entropy, then keys can be inserted in an order that will cause nodes of similar sizes to cluster together, bringing the time complexity closer to O(n).

What was the hardest bug or design decision you encountered, and how did you resolve it?

The hardest design decision was how to store the next pointers. The easiest solution was a vector, but this meant that when creating a new node, there would be a heap allocation for that node, and a separate heap allocation for the vector, which was really inefficient and was going to show up in the experiments. I thought of instead just declaring an array with max_levels size, but it seemed really inefficient to allocate 32 pointers when in most cases there would only be heights of 1 or 2.

I resolved this by comparing with Aaron's implementation, and asking Claude for advice on what would be the best implementation. It suggested manually allocating the memory. I wanted to avoid it because I wasn't well experienced in manual memory management, but decided it would be the best for the implementation, so I learnt all the extra memory concepts with help of implementation with Claude Code.

Laith Alneemy 24958454

If you did this implementation again from scratch, what would you do differently?

I would've included some way to reduce unnecessary key comparisons. In the experiments, it was shown that around half the time, when a node was checked which caused the level to drop, the same node is checked again when the node is dropped, which with expensive keys like long strings, wastes unnecessary comparisons. I would experiment with storing the previous pointer temporarily, and comparing the pointers before the values, and dropping levels immediately if the pointers are identical, which I suspect would be much cheaper than key comparisons in many situations.

What's one thing about your algorithm that you didn't fully understand before starting, and now do?

I didn't understand how randomness could be useful in such a data structure like a sorted linked list. I now realise how randomness guards against input/erasure order being a relevant factor. For instance, without randomness, something like a binary search tree that doesn't have a way to balance itself has its performance reliant on input and erase order. Balancing requires complex logic in implementation. By using randomness, we decouple order of input, as well as remove the requirement for balancing logic, because as long as the random information remains private, there's patterns or way to exploit data structures that utilise randomness, apart from getting severely unlucky.